Ziyang Liu

Singapore | +65 8591 7024 | ziyang.liu.r@outlook.com | ziy.bio | GitHub/ziyangliu-666 | LinkedIn/ziyangliu666

Education

National University of Singapore Aug 2026 – Nov 2027
M.Tech. in Software Engineering *Singapore*

University of Electronic Science and Technology of China Sep 2020 – Jun 2024
B.Eng. in Software Engineering *Chengdu, China*

Experience

Virtualization R&D Engineer Jul 2024 – Sep 2025
SmartX *Chengdu, China*

- **Migration product:** led the design of V2V OS 1.6.0 to 1.7.2, which moves virtual machines from VMware to SMTX OS and between SMTX clusters. Built across the end-to-end path: VMware CBT snapshots, a Python control plane, and a C data-transfer core exposed as a shared library. The product now serves 10,000+ production VMs, including core financial workloads.
- **Transfer performance:** replaced the serial read-then-write path with asynchronous chunked reads and writes across a bounded queue, using 8 reader and 8 writer workers. Handled partial I/O, cancellation and first-error propagation across the pipeline. Increased sustained throughput from 70 MB/s to 290 MB/s.
- **VMware compatibility:** isolated each migration in its own process, with one VDDK version loaded per process. This removed shared-process SDK symbol and global-state conflicts, contained crashes to one task, and expanded support across vSphere releases.
- **Destination-platform support:** prepared migrated VMs to boot on new virtual hardware through offline driver injection. Replaced the NFS disk-attachment path after hung mounts blocked guest preparation, using iSCSI block devices instead; then adapted the pipeline to SMTX ELF disaggregated compute-storage clusters.
- **In-guest agent:** rebuilt VMTools, replacing shell scripts over QGA guest-exec with native QEMU Guest Agent commands and resource probes. Cut response time from 3 s to under 50 ms and yearly escalations from 40 to under 5.
- **Consistent snapshots:** made snapshots filesystem-consistent rather than only crash-consistent. The control plane coordinates VMTools and fsfreeze to quiesce the guest filesystem before the storage snapshot, then thaws it after the snapshot boundary.
- **Cross-cluster live migration:** built a block-level data check triggered by a libvirt hook after switchover. It snapshots source and destination disks through the storage API, reads and hashes them block by block, and keeps compare order in sequence queues on both sides so verification runs concurrently without blocking migration.
- **Kernel and QGA debugging:** used strace to find a UOS kernel creating pidfds without reaping them, hanging customer VMs; the fix became a financial-sector advisory and earned a written commendation. Also traced truncated host-to-guest JSON to old kernels not re-polling the QGA serial fd.

Virtualization R&D Intern Oct 2022 – Jul 2024
SmartX *Chengdu, China*

- **Migration appliance:** built V2V OS 1.2.0 to 1.6.0 as a self-contained virtual appliance on openEuler for government procurement. Rebuilt and packaged its kernel with the filesystem and block-device features required to inspect and modify newer guest images.
- **Low-downtime migration:** extended the workflow from one full copy plus a long outage to an initial copy followed by repeated incremental passes while the VM keeps running. VMware Changed Block Tracking (CBT) identifies blocks dirtied after each snapshot, leaving only a small final delta for the stopped-VM switchover.
- **Packaging and release:** wrote the rpm.spec for V2V and its custom kernel. Built CI/CD that assembles both into a bootable ISO and a Packer pipeline that produces the vSphere OVF appliance.
- **Offline driver injection:** made migrated guests boot on virtio hardware by rebuilding Linux initramfs images and replaying Windows registry changes captured with procmon around a virtio-win installation. Supported both MBR and UEFI, with disk snapshots and an IDE fallback for recovery.
- **Guest coverage:** carried driver injection and VMTools across nine Linux distributions, including openEuler, UOS and Kylin, plus Windows Server 2012–2025. Maintained per-OS handling for boot layout, package tools, service startup and guest-agent installation.

Systems Integration Development Intern Jun 2022 – Sep 2022
ByteDance (Feishu) *Chengdu, China*

- **Workflow integration:** built Go and Gin services connecting recruiting workflows to external OA systems, with MongoDB holding sync and retry state.
- **API hardening:** added authentication, rate limiting and a regression suite across the integration endpoints.

Project

Maintainer Sep 2026 – Present

FastMM, market-making engine in C++20 *ziy.bio/FastMM*

- **Live validation: #1 on 2 pairs** and **7.5M USDT** in one week in Binance's Spot Liquidity Provider Program (LinkedIn post). Weekly volume grew from 2.9M on $10K to 7.5M on $36K. Top 10 on **38 of 77** pairs.
- **Hot path:** designed the engine around one trading thread that owns all state, so the hot path needs no locks; other threads reach it through lock-free queues. No memory is allocated after start-up. In the simulator benchmark, market data to order takes 159 ns.
- **Live latency fix:** found orders waiting 2 to 6 ms inside the engine in live trading, because each balance update re-priced all 85 pairs. Changed re-pricing to once per batch, cutting balance handling p99 from 23.8 to 1.2 µs.
- **CPU contention:** found six busy-waiting threads competing for the 2-vCPU live-trading cloud host. Switched idle threads from spinning to backing off, cutting latency p50 from 2.5 to 0.23 ms and p99 from 7.0 to 0.49 ms.
- **Market data:** switched Binance market data from JSON to its binary format, cutting decode time from 680 to 51 ns. Wrote connectors for 7 crypto exchanges and Nasdaq, plus AF_XDP and DPDK receive paths.
- **Backtest accuracy:** made backtest and live run the same strategy code, with deterministic replays. Replayed with real arrival times and latency, raising backtest adverse fills from 2.6% to 16.0%, against 17.2% live.
- **Python:** added a Python strategy interface, with Numba compiling the fast parts to machine code. Published it on PyPI as fastmm-engine.
- **Dashboard:** built bndesk (source, demo), a Rust dashboard for live trading: P&L by source, fill quality, quotes and inventory for every sub-account. Made the server send only what changed. Live demo on request.

Research

Researcher Sep 2025 – Aug 2026

HKUST (Guangzhou) *Guangzhou, China*

Under review

- Z. Liu, Y. Shi. A Simpson's Paradox in In-Context Learning Curves. ICLR 2027 submission. The average loss falls but rises within every token-occurrence group; domain explains 99% of the standard score's variance, versus 0.2% for model size.
- Y. Shi, Z. Liu, K. Luo, N. Tang. A Living In-the-Wild Benchmark for Measuring the Static-Benchmark Gap in AI-Generated Video Detection. ICLR 2027 submission. On VidTide's 21,504 clips, five detector families fall from published 93–96% AUROC to 52–71%.
- Y. Shi, K. Luo, Z. Liu, N. Tang. Neuro-Symbolic Forensic Reasoning for Open-Generator AI-Generated Video Detection. ICLR 2027 submission. Without training detector weights, FOVEA reaches 97.39% accuracy and 0.99 AUROC on ten unseen generators.

In preparation

- **Committed SAE-feature traces:** made a hosted API prove it served the model it advertised. Fingerprinted every answer with a sparse-autoencoder feature trace, committed to a Merkle root before any check is requested. Cheating then means failing the check or paying for the real model anyway. Caught all 17 substitutes across three backbones, under 2.1% overhead.
- **Cooperative memory paging:** kept a long chat usable after it outgrows the context window. Checked first whether the model signals a missing fact through token-level uncertainty. It does not, and answers more confidently without it. Left an ∼8-token keyword bookmark in place of each dropped section, plus a recall() tool for the full text. Beat five methods on LoCoMo, four LLM judges agreeing at $p = 0.017$.
- **Copy-as-decode:** cut the cost of re-emitting unchanged text when an LLM edits a file. Had it name the input lines to copy, spliced into the KV cache by one parallel prefill. Speculative decoding, except the draft is the input document and a grammar guarantees acceptance. Copied text is 74 to 98% of an edit, bounding the speedup at 13×.

Technical Skills

Languages: Python, C++, C, Rust, Go, Shell
Networking: TCP/UDP multicast, epoll, io_uring, AF_XDP, DPDK, tcpdump
Linux/UNIX: processes and threads, virtual memory, NUMA, CPU affinity, cgroups, strace/perf/gdb
Virtualisation & migration: QEMU/KVM, libvirt, Virtio, VMware CBT/VDDK, QGA, live migration
LLM systems: memory tiering, retrieval and reranking, ReAct, MCP, KV cache
Build & operations: CMake, GitHub Actions, Jenkins, Docker, RPM/DEB, Packer
Databases: MySQL, Redis, MongoDB, SQLite

Honours

National Third Prize, China Collegiate Computer Design Competition (2022) | UESTC Scholarship (2020–21)